HACKATHON PROBLEM STATEMENT

Real-Time Price Monitoring System for Bearing Distributors on Amazon India

Company: Nisargasoft

Domain: E-commerce Intelligence / Data Scraping / Competitive Pricing Analytics

Track Information

Track: Data Engineering / Web Scraping / Market Intelligence

Difficulty: Intermediate – Advanced

Core Domains: Web Scraping, Data Pipelines, Market Analytics, Competitive Pricing

Primary Languages: Python (recommended)

Key Libraries: Scrapy, BeautifulSoup, Selenium/Playwright, Pandas, Proxy rotation tools

Deliverable: Working scraping pipeline + price monitoring dashboard + analytics report

1. Executive Summary

Industrial bearing distributors in India frequently sell products such as SKF bearings on large e-commerce platforms like Amazon.in. These distributors compete heavily on price because multiple sellers often list the exact same bearing product under the same ASIN. This challenge focuses on building an automated system capable of monitoring pricing behavior across sellers in real time. The system should collect product listings, detect price fluctuations, identify default sellers for specific buyer locations, and provide actionable insights to help distributors adjust pricing strategies and maintain competitive market positioning.

2. Background & Context

Industrial bearing distributors in India compete aggressively on platforms like Amazon.in. Products such as SKF ball bearings are often sold by multiple distributors under the same product listing (ASIN). Sellers frequently adjust prices to undercut competitors and capture the default Buy Box position.

A system capable of continuously monitoring prices, seller positions, and availability can provide significant competitive advantage. Such intelligence allows distributors to respond quickly to price drops, avoid unnecessary margin loss, and optimize inventory planning.

3. Problem Statement

Design and develop an automated price monitoring system that tracks bearing products (such as SKF models) across multiple sellers on Amazon India.

The system should:

- Search Amazon.in for bearing products such as 'SKF bearing [model]'.
- Identify and extract product ASINs from search results.
- Monitor prices from multiple sellers offering the same ASIN.
- Track the seller currently appearing as the default Buy Now seller.
- Detect price fluctuations, promotions, or stock-related price spikes.
- Store historical price data for analysis and trend detection.
- Provide a dashboard for distributors to view competitor pricing behavior.

4. Business Value

Real-time monitoring allows distributors to track competitor quotes and maintain optimal profit margins while remaining competitive in the market.

For example, if a Chennai-based distributor observes a rival dropping prices for a popular SKF bearing, they can adjust pricing or run targeted promotions to stay competitive.

The system also helps identify market leaders and price trends, allowing distributors to optimize inventory for high-demand items and anticipate market demand.

5. Detecting Default Sellers by Location

Amazon prioritizes sellers whose fulfillment centers are geographically closer to the buyer's location. For example, buyers in Chennai may see sellers with FBA warehouses in Tamil Nadu displayed as the default 'Buy Now' seller.

The system should simulate buyer locations by configuring request headers or proxies mapped to Tamil Nadu PIN codes. This enables detection of which seller appears first for specific geographic locations.

The tool should also track situations where the default seller's price changes due to stock shortages, supply issues, or temporary promotions.

6. Technical Implementation Outline

Suggested architecture:

- Use Python Scrapy framework for scalable crawling.
- Search Amazon.in for queries like 'SKF bearing [model]'.
- Paginate search results using query parameters such as &page=.
- Extract ASINs, product titles, seller names, and prices from listing pages.
- Visit product detail pages to extract full seller offers and FBA status.
- Track which seller holds the default Buy Box position.

- Store scraped data in a structured database.
- Use scheduled jobs (cron or task scheduler) to run monitoring periodically.

7. Data Collection & Infrastructure

- Implement IP rotation using proxy services such as ScrapeOps.
- Use a pool of rotating IP addresses (e.g., 100+ proxies) to avoid blocking.
- Implement headless browsing where required.
- Handle pagination, dynamic page loading, and seller offer sections.
- Store collected data in databases such as PostgreSQL or MongoDB.

8. Analytics & Dashboard

- Display real-time competitor price comparison.
- Show historical price trends for each ASIN.
- Identify price leaders and frequent price changes.
- Highlight undercutting strategies used by competitors.
- Provide alerts when competitors drop prices significantly.

9. Evaluation Criteria

- Scraping reliability and accuracy of extracted product data.
- Ability to detect ASINs, sellers, and price changes correctly.
- System robustness against blocking and anti-bot mechanisms.
- Efficiency of proxy rotation and crawling strategy.
- Accuracy in identifying default Buy Box sellers.
- Quality and usability of the analytics dashboard.
- Ability to store and analyze historical pricing trends.
- Code quality, documentation, and reproducibility of the system.

10. Deliverables

- A working web scraping system that monitors bearing prices on Amazon India.
- A data storage pipeline for historical price tracking.
- A dashboard showing competitor pricing insights.
- Documentation describing scraping architecture and anti-blocking strategies.
- Instructions for deployment and execution.